

Sukkur Institute of Business Administration University
Department of Computer Science

CS304 – Software Quality Engineering
BS – V (Software Engineering) – "A"
Fall 2026

Instructor: Dr. Khursheed Ali

Final Project Proposal

Bookstore Management System

Submission Profile

Name: Muhammad Anas Qadri	**CMS ID:** 053-24-0016
Name: Isra Memon	**CMS ID:** 053-24-0017
Name: Muhammad Waleed	**CMS ID:** 053-24-0100

Table of Contents

1. Project Name 3
2. Group Members (Section A) 3
3. Problem Statement 3
4. Brief Project Description 3
5. Project Objectives 3
   5.1 Product Objectives 3
   5.2 Software Quality Engineering Objectives 4
6. Project Scope 4
   6.1 In Scope 4
   6.2 Out of Scope 4
7. Target Users 5
8. Key Features 5
9. Quality Focus and Testing Approach 6
10. Tools/Technologies to Be Used 6
11. Group Responsibilities 7
12. Preliminary Project Schedule 8

1. Project Name

Bookstore Management System (BMS) – a desktop application for a local bookstore.

2. Group Members (Section A)

1. Muhammad Anas Qadri (CMS ID: 053-24-0016)
2. Isra Memon (CMS ID: 053-24-0017)
3. Muhammad Waleed (CMS ID: 053-24-0100)

3. Problem Statement

Qadri Book Store, located in Shahi Bazaar, Rohri, sells school books, stationery, and notebooks to schools placing bulk orders and to individual walk-in customers. Most of its daily operations are still handled manually.

Stock is tracked in notebooks, so stock levels are often inaccurate. At the start of each academic session this causes stock shortages, delayed bulk school orders, and billing difficulties. Supplier purchases and sales records are only loosely tracked, which increases human error and gives the owner no clear view of the business.

The store needs a simple digital system that runs offline on its existing counter PC, since internet access is unreliable, and fits the way the store already works.

4. Brief Project Description

The Bookstore Management System (BMS) is a standalone desktop application that digitizes a single bookstore's daily work: inventory management, walk-in sales with automatic billing and receipts, school bulk orders and quotations, supplier purchase orders, and sales and inventory reports.

Access is controlled by login with two roles: Admin (store owner/manager) with full access, and Staff (counter staff) for sales and basic inventory look-ups. All data is stored locally on the store's computer, with backup and restore support.

5. Project Objectives

5.1 Product Objectives

- Reduce manual work and human errors in daily bookstore operations
- Improve inventory tracking, including low-stock alerts
- Handle school bulk orders from registration to quotation and order tracking
- Maintain accurate records of sales, purchases, and supplier orders
- Speed up billing with printed receipts

5.2 Software Quality Engineering Objectives

- Define functional and non-functional requirements with measurable acceptance criteria
- Apply reviews and quality assurance techniques to requirements, design, and code
- Design and execute unit, integration, system, and user acceptance tests using black-box techniques (equivalence partitioning, boundary-value analysis, positive and negative testing)
- Identify, document, classify, and track defects through retesting
- Evaluate product quality using metrics based on real, collected evidence
- Use GitHub for version control, issue tracking, and collaboration

6. Project Scope

6.1 In Scope

- Inventory management of books, stationery, and other items (add, update, delete, search, stock levels, low-stock alerts)
- Point-of-sale for walk-in customers, with billing, automatic stock update, and receipts
- School registration, bulk orders, quotations, and order status tracking
- Supplier management, purchase orders, and recording of goods received
- Sales and inventory reports
- User authentication, role-based access (Admin/Staff), user management, and data backup and restore
- Printing of receipts and reports on the store's thermal printer

6.2 Out of Scope

- Online shopping or e-commerce functionality
- External payment gateways (online banking, mobile wallets); all sales are cash-based
- Multi-branch or franchise management
- Advanced analytics or AI-based forecasting
- Mobile application development

7. Target Users

User	Type	Role in the System
Admin (Store Owner / Manager)	Primary user	Full access: manages products, suppliers, purchase orders, school orders, reports, user accounts, and backup/restore.
Staff (Counter Staff)	Primary user	Processes daily sales, generates receipts, and searches/views product information.
Schools	External	Place bulk orders and receive quotations prepared by the Admin.
Walk-in Customers	Secondary	Buy items at the counter and receive printed receipts.
Suppliers	External	Supply books and stationery and receive purchase orders.

8. Key Features

Module	Main Features
User Authentication	Login and logout; role-based access for Admin and Staff
Inventory Management	Add, update, delete, and search products; unique SKU per product; view stock levels; low-stock alerts
Sales Management (POS)	Create a sale, add items, automatic total calculation, automatic inventory update, receipt generation
School Order Management	Register schools, create bulk orders, generate quotations, track order status
Supplier Management	Add suppliers, create purchase orders, record goods received
Reporting	Sales reports and inventory reports
System Administration	Manage user accounts, data backup and restore, print receipts and reports

9. Quality Focus and Testing Approach

These targets come from the non-functional requirements in the SRS and will be tested as measurable acceptance criteria.

Quality Attribute	Target / Criterion
Performance	Search and button response within 3 seconds; a sales transaction processed within 5 seconds; at least 100–200 transactions per day without errors
Accuracy	Correct totals, stock updates, and reports
Reliability / Safety	Confirmation before deleting data; invalid or incomplete entries rejected; critical operations saved automatically; backup supported; clear error messages
Security	Login required; role-based access; protection of stored data; logging of important activities
Usability	Simple interface usable with basic computer skills; clear labels; success, error, and confirmation messages
Documentation	User manual, quick start guide, on-screen instructions, and tooltips

10. Tools/Technologies to Be Used

The proposed stack combines a Java backend with a web-based frontend to build an offline-first Windows desktop application, supported by automated testing and code-quality tools.

1. Java (JDK 25 LTS) – Primary language for business logic, data validation, and application services.
2. Spring Boot – Backend framework for REST APIs, validation, and business logic.
3. Next.js with TypeScript – Frontend framework for a responsive, type-safe, component-based interface, configured for desktop deployment.
4. HTML5 and CSS3 – Structure and styling of application screens, forms, and tables.
5. Electron – Packages the frontend as a standalone Windows desktop application.
6. Node.js and npm – Runtime and package management for frontend and Electron development.
7. SQLite – Embedded database for local storage of products, stock, sales, and customer records, with no separate database server.
8. Spring Data JPA / Hibernate – Data access between the Java backend and SQLite.
9. Maven – Build automation and dependency management for the backend.
10. Java Print Service / Electron Printing API – Printing of receipts, invoices, and reports, depending on the target printer.
11. JUnit 5 and Mockito – Unit testing of business logic and isolation of dependencies.

12. Vitest and React Testing Library – Testing of frontend components and user interactions.
13. Windows 11 – Target operating system of the store's counter computer, subject to compatibility testing.
14. Git and GitHub – Version control, issue and defect tracking, pull-request reviews, and collaboration.
15. Draw.io / PlantUML – Architecture, entity-relationship, UML, and sequence diagrams.

The application will run locally without internet for routine store operations. SQLite will handle local storage, Spring Boot will handle business rules and data access through a REST API, and Electron will host the Next.js frontend (for example, as a static export). A modular architecture will separate the user interface, business logic, and data-access layers, and the final configuration will be evaluated against the target computer's memory, processing power, and printing needs.

11. Group Responsibilities

All members will work together across the software development life cycle, including requirements analysis, design, frontend and backend development, database integration, testing, documentation, code reviews, and deployment. Tasks will be shared and rotated fairly.

Each member also holds a primary coordination role, which does not limit their participation in other areas.

Member	Primary Role	Responsibilities
Muhammad Anas Qadri	Project coordination and progress tracking	SRS preparation, architecture and database design, integration, deployment, and GitHub repository and branch management.
Isra Memon	UI and Feature Developer	UI/UX design, feature development, and database integration.
Muhammad Waleed	QA Lead and Tester	Test planning, functional and integration testing, and defect resolution.

12. Preliminary Project Schedule

The schedule is counted in weeks from proposal submission and will be aligned with the instructor's submission dates.

Week	Activity	Deliverable
Week 1	Finalize the proposal and group setup; gather and analyze requirements; define functional and non-functional requirements, acceptance criteria, and use cases; prepare the initial SRS; set up the GitHub repository and issue tracking.	Project Proposal and Software Requirements Specification (SRS)
Week 2	Finalize the SRS; design the architecture, database schema, UML diagrams, and UI prototypes; define coding standards, quality objectives, review checklists, risk procedures, and the testing strategy; prepare initial test cases and the traceability matrix.	Software Design Document, GitHub Repository, Software Quality Assurance Plan, and Initial Test Plan
Week 3	Implement core features iteratively; integrate the Next.js frontend, Spring Boot backend, and SQLite database; conduct code reviews, unit testing, and integration testing; record and resolve defects; maintain documentation.	Working Application (Beta), Source Code, Unit and Integration Test Results, and Updated Test Documentation
Week 4	Complete remaining features and integration; conduct system, user acceptance, and regression testing; fix defects; measure quality metrics; finalize documentation and the project report; prepare the presentation, live demonstration, and individual viva.	Final Software Product, Test Execution and Defect Report, Software Quality Metrics Report, Final Project Report, and Demonstration